B.Sc. (Hons) in Software Development

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

3D Model Platform: Template-Driven STL Generation and Slicing

By
Michal Zdziubany

April 26, 2026

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Contents

1 Introduction **1**
1.1 Background and Context . . . 1
1.2 Problem Definition . . . 2
1.3 Aim and Objectives . . . 3
1.4 Scope and Delimitations . . . 4
1.5 Research Questions . . . 4
1.6 Chapter Summary . . . 5

2 Methodology **6**
2.1 Methodological Overview . . . 6
2.2 Software Development Methodology . . . 6
2.3 Research Design . . . 7
2.4 Research Methodology . . . 7
2.5 Validation Strategy . . . 8
2.5.1 Iteration Structure . . . 9
2.5.2 Implementation Phases . . . 9
2.6 Technical Stack and Rationale . . . 10
2.7 Data Collection and Evaluation Procedure . . . 11
2.7.1 Evaluation Dimensions . . . 11
2.8 Validity, Reliability, and Limitations . . . 12
2.9 Chapter Summary . . . 12

3 Technology Review **13**
3.1 Chapter Overview . . . 13
3.2 Review Scope and Criteria . . . 13
3.3 Existing Platform Context . . . 14
3.4 Parametric Geometry Generation . . . 14
3.5 Slicing and Print Preparation . . . 15
3.6 Web Application Architecture . . . 15
3.7 Deployment and Reproducibility . . . 16
3.8 Technology Selection Summary . . . 16

4 System Design **17**
4.1 Architectural Overview . . . 17
4.1.1 Layered System View . . . 19
4.2 Design Requirements . . . 19
4.2.1 Architecture by Concern . . . 20
4.3 Key Backend Components . . . 20
4.3.1 Routing Layer . . . 21
4.3.2 Service Layer . . . 22
4.4 Data and File Flow . . . 22
4.4.1 Workflow Algorithm . . . 23
4.5 Template and Parameter Model . . . 24
4.5.1 Built-in Templates . . . 24
4.5.2 User Templates . . . 25
4.6 Template Strategy . . . 25
4.7 Frontend Interaction Design . . . 26
4.7.1 User Interaction Sequence . . . 27
4.8 Error Handling and Safety Considerations . . . 27
4.9 Deployment Design . . . 28
4.9.1 Runtime Configuration . . . 29
4.10 Design Rationale . . . 29
4.11 Chapter Summary . . . 29

5 System Evaluation **31**
5.1 Evaluation Approach . . . 31
5.1.1 Evaluation Scope and Boundaries . . . 31
5.2 Evaluation Criteria . . . 32
5.3 Objective-Based Results . . . 33
5.3.1 Proposal Objective Completion . . . 34
5.4 Comparative Positioning Against Existing Platforms . . . 35
5.5 Research Question Reflection . . . 37
5.5.1 Direct Research Question Answers . . . 37
5.6 Representative Test Scenarios . . . 38
5.7 Strengths . . . 39
5.8 Limitations and Risks . . . 39
5.9 Critical Reflection . . . 40
5.10 Chapter Summary . . . 40

6 Conclusion **41**
6.1 Summary of Achievements . . . 41
6.2 Key Contributions . . . 42
6.3 Answer to Research Questions . . . 42

6.4 Lessons Learned 43
6.5 Limitations 44
6.6 Future Work 44
6.7 Final Remarks 45

Appendices **47**

A Github Repository **48**

B Screencast **49**

List of Figures

4.1 Overall system architecture of the 3D Model Platform. 18

List of Tables

4.1 Component-level view of the platform architecture. 20
4.2 Simplified template metadata model used by the platform. 24
4.3 Representative failure handling points in the architecture. 28

5.1 Proposal objectives completion status 34
5.2 High-level positioning of the project platform against selected industry platforms 36
5.3 Representative Evaluation Scenarios 38

Abstract

3D model preparation for Fused Deposition Modeling (FDM) remains fragmented across multiple specialized tools, requiring users to context-switch between parametric design, mesh editing, slicing, and file management. This project addresses that workflow friction through a template-first web platform that unifies parametric model generation, STL output, and G-code slicing in one integrated system. The platform combines Next.js, FastAPI, OpenSCAD, and CuraEngine with Docker containerization to ensure reproducible, deterministic workflows.

Rather than replacing general-purpose CAD systems, the platform targets common parametric part families where workflow efficiency and repeatability matter most. Core objectives included designing reusable templates, implementing server-side STL generation, integrating profile-based slicing, supporting user template upload, and providing configurable printer profiles.

Evaluation demonstrates that the integrated platform successfully reduces repeated tool handoff and enables faster iteration for parametric parts. A modular backend architecture supports extensibility, while containerized deployment improves reproducibility. The study confirms that template-driven generation reduces setup overhead and that unified generation and slicing improve workflow continuity. Practical benefits depend on clear defaults, sensible parameter validation, and adequate user onboarding.

The project demonstrates that targeted integration of established open-source tools can effectively reduce friction in additive manufacturing workflows when the problem is scoped carefully and architecture prioritizes clarity.

Keywords: additive manufacturing, FDM slicing, parametric design, workflow integration, template-driven generation

Chapter 1

Introduction

1.1 Background and Context

Fused Deposition Modeling (FDM) remains one of the most accessible additive manufacturing techniques because it offers a practical balance between cost, availability, and functional output. For many everyday modelling tasks, the important challenge is not the printer itself but the preparation workflow that leads to a usable file. Users typically move from sketching an idea, to creating geometry, to exporting a mesh, to configuring slicer settings, and finally to producing G-code. Each stage is individually manageable, but the full process can become slow and error-prone when it must be repeated frequently.

In this workflow, STL and G-code serve distinct purposes. STL is a common mesh format used to describe the surface geometry of a model in a form that can be exchanged between modelling and printing tools. It does not store printer settings or material behaviour; instead, it provides the geometric input required for later processing. G-code is the machine instruction format used by most 3D printers. It contains the movement and process commands that tell the printer how to build the part layer by layer.

The conversion between these formats is performed through slicing. Slicing is the process of taking a 3D model, applying printer and material settings, and converting the geometry into a sequence of layer-by-layer instructions suitable for fabrication. In practical terms, the slicer determines how the part will be printed, including settings such as layer height, infill, temperatures, speeds, and support generation. Because slicing translates model geometry into executable print instructions, it is one of the most important stages in the preparation workflow.

That difficulty is especially apparent in student, hobbyist, and small-project settings where the goal is often to produce a simple printable part rather than develop a complete CAD model. In these contexts, the overhead of switching

between tools can outweigh the actual design work. Small parameter changes may require repeated reconfiguration, and local environment differences can affect whether the same model can be reproduced consistently on another machine. As a result, the workflow itself becomes a source of friction.

This project addresses that problem through a web-based, template-driven 3D model platform that combines parameter input, STL generation, and G-code slicing in one workflow. The system brings together a Next.js frontend, a FastAPI backend, OpenSCAD for deterministic geometry generation, and CuraEngine for slicing, with Docker used to improve reproducibility across environments [1, 2, 3, 4, 5]. Rather than trying to replace full CAD systems, the platform focuses on common parametric parts where speed, repeatability, and usability matter more than unrestricted modelling freedom. In that sense, it is a deliberately constrained tool designed for practical workflow efficiency.

The work presented here follows directly from the original project aim: lower the barrier to parametric model generation while keeping clear control over slicing and outputs. In that sense, the platform is both a software artifact and a practical case study in how workflow integration can make additive manufacturing preparation easier to use.

1.2 Problem Definition

Conventional FDM preparation workflows require repeated context switching and a degree of tool-specific knowledge that is unnecessary for many common tasks. This becomes a practical limitation when users need to generate a part quickly, adjust a few dimensions, and produce a printable output without rebuilding the entire design process from scratch. The issue is particularly visible in project-based learning, prototype development, and small workshop settings where time, clarity, and consistency are important.

From a software engineering perspective, the problem is broader than geometric modelling alone. It is also an integration problem. A user needs a dependable path from validated parameters to printable machine instructions, but the usual workflow distributes that responsibility across separate tools with different assumptions about file formats, settings, and execution environment. That separation increases the chance of mistakes and makes the process harder to reproduce.

Consider a concrete example: a student needs to print five variants of a parametric bracket with different hole sizes. Using conventional tools, the workflow is: open CAD software (tool 1), create the first bracket, export to STL, open slicer (tool 2), load the STL, configure print settings, export G-code, repeat this for each variant, managing files across the desktop and downloads folder. If the student needs to adjust the hole size on all five parts, they must repeat the entire process.

If the student is using a different computer, they must reconfigure the slicer settings from scratch. If a setting is misremembered or mistyped, the print may fail and material is wasted.

The core gap addressed in this project is the mismatch between user intent and toolchain structure. Users typically want to generate a printable part quickly, while the software stack expects them to configure several independent systems. This work investigates whether an integrated, template-first platform can reduce that mismatch while still preserving enough configurability to remain useful beyond trivial examples.

The core problem investigated in this project is:

> How can an integrated, template-first web platform reduce friction in producing printable FDM outputs while preserving practical configurability and output reliability?

1.3 Aim and Objectives

The aim of this project is to design, implement, and evaluate a practical software platform that unifies parametric model generation and slicing for FDM printing.

The project objectives are:

- design a reusable template catalog for common parametric part families,
- implement server-side STL generation from parameterized OpenSCAD templates,
- integrate profile-based slicing to G-code using CuraEngine,
- support user-defined template upload and execution,
- provide configurable printer and slicing profile defaults,
- evaluate technical reliability and workflow usability against defined criteria.

These objectives were scoped to balance technical ambition with deliverability in a student project. The emphasis is on integration quality rather than broad CAD capability. In practical terms, success is defined by reliable execution of the complete user journey, from template selection to printable output, rather than by including every possible modelling feature.

Two proposal ideas, natural language input and OctoPrint integration, were considered but deferred because they would have expanded the project beyond the core objective of demonstrating a stable template-to-output workflow.

1.4 Scope and Delimitations

This project covers software architecture, backend service design, frontend interaction design, template processing, artifact generation, and deployment concerns for a web-accessible FDM preparation toolchain. It includes support for built-in templates, user-uploaded templates, profile-aware slicing, and file-oriented job outputs.

The platform does not include direct printer firmware control, or advanced freeform CAD capabilities such as assembly constraints and topology editing. These exclusions are deliberate. They keep the evaluation focused on workflow integration, simplicity and software reliability rather than broad printer automation or general-purpose modelling.

The rationale for these boundaries is practical scoping rather than fundamental limitation. A full additive manufacturing system would be a significantly larger project. Printer control and monitoring can be added later without fundamentally changing the architecture. Similarly, supporting only parametric templates means the system remains fast and predictable, users needing freeform modeling can continue using CAD software while users with common parametric parts can use this platform.

A further delimitation is that evaluation is primarily software-centric rather than manufacturing metrology-centric. The project verifies that valid G-code is produced and that the workflow behaves consistently, but it does not attempt a full physical print-quality campaign across multiple materials and machine combinations. This reflects the project focus on demonstrating feasibility of the integrated workflow rather than performing exhaustive manufacturing science validation. In practice, this means the platform produces G-code that is structurally sound, but whether that G-code produces superior parts compared to manually-sliced files is outside the evaluation scope.

1.5 Research Questions

The study is guided by the following research questions:

1. To what extent does a template-first workflow reduce effort in preparing printable files?

2. How effectively can integrated generation and slicing improve end-to-end iteration time?

3. What software design trade-offs emerge between simplicity, extensibility, and configurability?

These questions connect implementation decisions with measurable project outcomes. The first question targets workflow efficiency and user effort. The second focuses on technical pipeline integration and response behaviour. The third addresses the architectural tension between simplicity, extensibility, and configurability.

The later chapters answer these questions through implementation analysis, scenario-based evaluation, and reflective comparison of the resulting system design.

1.6 Chapter Summary

This chapter set out the context, problem, and scope of the project, and introduced the aim, objectives, and research questions that guide the rest of the dissertation. Chapter 2 explains the development and evaluation methodology. Chapter 3 reviews the technologies and related platforms that informed the implementation. Chapter 4 details system design and implementation decisions. Chapter 5 evaluates outcomes against the stated objectives and research questions. Chapter 6 closes with contributions, limitations, and future work.

Overall, the project is framed as an applied software engineering study of an integrated additive manufacturing workflow. The platform is treated as a practical tool, with the emphasis on reducing preparation friction while still retaining useful flexibility.

Chapter 2

Methodology

2.1 Methodological Overview

This project adopts an applied software engineering methodology centred on designing, implementing, and evaluating a working software artifact. That approach is appropriate because the goal is an integrated workflow rather than a single algorithm. The main task was to combine template rendering, STL generation, slicing, and user-facing interaction into one coherent system.

The methodology focuses on building a system that can be examined as an artifact. Quality is assessed through implementation evidence, repeatable workflow scenarios, and direct comparison between intended and observed behaviour. The goal is not just to show that the platform works, but to explain why the design choices support the intended workflow.

This approach fits the problem because the research question is about workflow integration, usability, and reliability rather than a novel computational algorithm. A design-and-evaluate strategy is therefore more suitable than a purely experimental or purely descriptive method. It allows the project to test whether the platform reduces friction in practice while still preserving enough configurability for real use.

2.2 Software Development Methodology

The software was developed using an iterative, incremental approach informed by agile principles [6]. Work was organised into short cycles that each delivered a usable slice of functionality. This allowed the project to evolve through successive refinements rather than through a single large implementation step.

Each cycle followed the same general pattern: define a focused feature objective, implement the backend and frontend changes needed for that objective, test

the integrated workflow, and then refine the implementation based on observed behaviour. This approach was particularly useful because the project involved several tightly connected layers, including frontend interaction, backend orchestration, template processing, and external CLI tooling.

The development methodology was appropriate for three reasons. First, it allowed early identification of integration issues, such as mismatches between frontend payloads and backend request models. Second, it supported rapid feedback on user-facing behaviour, which was important for a workflow-oriented system. Third, it reduced the risk of large late-stage changes by keeping the system operational at the end of each cycle.

The implementation was also structured around vertical slices rather than isolated component work. Instead of completing all backend logic before frontend integration, each iteration aimed to produce one end-to-end capability, such as template listing, STL generation, or slicing. This made it easier to validate the system in realistic conditions and to ensure that the final platform behaved as a coherent workflow rather than as a set of disconnected modules.

2.3 Research Design

The study follows a design-and-evaluate paradigm:

1. identify workflow problems in conventional FDM preparation,
2. design and implement an integrated software solution,
3. evaluate the implementation against predefined objectives and scenarios,
4. critically reflect on limitations, threats to validity, and transferability.

This structure ensures that implementation is not treated as an end in itself. Each feature is linked to a research objective and later assessed against observable evidence. The design phase was guided by explicit requirements, while the evaluation phase examined whether those requirements were met under representative usage conditions.

To preserve coherence, requirements and research questions were revisited after each implementation phase. This reduced feature drift and kept implementation effort aligned with the project aim.

2.4 Research Methodology

The research methodology is qualitative and evaluative, supported by scenario-based functional evidence. This work does not attempt to measure the platform

through large-scale user trials or statistical benchmarking. Instead, it investigates whether the implemented artifact meets its stated objectives in representative use cases.

This choice aligns with the project focus on software feasibility, workflow improvement, and design trade-offs. These questions are best answered by observing the completed artifact under realistic tasks rather than isolating a single variable in a controlled laboratory setup. In short, the platform is evaluated as a complete workflow, not as an abstract model.

Evidence was gathered through functional checks, generated artifacts, endpoint behaviour, and reflective assessment of implementation decisions. The main emphasis was on traceability from user input to STL and G-code output. In practical terms, this meant evaluating whether the system could consistently perform the tasks it was designed to support, and whether the architectural choices made those tasks easier to deliver and maintain.

The approach combines qualitative and limited quantitative evidence. Qualitative evidence came from implementation notes, error cases, and observed workflow friction. Quantitative evidence came from pass/fail outcomes, repeatable task completion, and the successful generation of output files. Together, these provided a reasonable basis for judging whether the approach was valid for the problem under study.

2.5 Validation Strategy

Validation was carried out at three levels. First, individual endpoints and services were checked to confirm that basic request and response behaviour was correct. Second, complete workflow scenarios were run to confirm that the system behaved coherently across the frontend and backend. Third, generated output files were inspected to ensure that STL and G-code artifacts were produced as expected.

This layered validation strategy was important because the system depends on several connected components. A route can be syntactically correct but still fail in practice if template rendering, CLI invocation, or file handling is incorrect. Likewise, a workflow can appear functional in isolation but fail when the system is used end to end. The chosen validation method was therefore designed to reflect the actual operational risk of the project.

Each iteration ended with a stability check to confirm that previously completed workflows still functioned. This reduced regression risk as the system grew.

The multi-layer approach was particularly critical because failures at each layer manifested differently. API-level validation could catch malformed requests early, but could not detect subtle issues in parameter normalization or environment path resolution. Service-level testing confirmed that template rendering and CLI invo-

cation worked in isolation, but could still miss integration issues where one component's output became the next component's input with unexpected formatting. End-to-end workflow testing exposed these integration failures, making it the most representative validation method despite being the most time-consuming. Consequently, the validation strategy prioritized end-to-end testing after establishing baseline stability at lower layers.

2.5.1 Iteration Structure

Each cycle followed the sequence below:

1. define a vertical feature objective,
2. implement backend route and service logic,
3. connect frontend interaction and response handling,
4. execute scenario-focused validation,
5. refine based on observed friction or defects.

The vertical feature approach was selected to maximise end-to-end visibility. Rather than completing all backend work before frontend integration, each iteration aimed to deliver a usable slice of functionality across both layers. This exposed user-facing issues earlier and reduced late-stage integration surprises.

Validation activities included manual API checks, scenario execution through the interface, and inspection of generated STL and G-code files.

2.5.2 Implementation Phases

Major phases were:

- Phase 1: baseline architecture and built-in template catalog,
- Phase 2: OpenSCAD-driven STL generation pipeline,
- Phase 3: CuraEngine slicing with profile configuration,
- Phase 4: user template upload and metadata handling,
- Phase 5: multi-part workflows, preview enhancements, and integration hardening.

Phase boundaries were used as control points for scope decisions. Features that did not directly support the project research questions were deferred. This was especially important for larger extensions such as advanced CAD editing or broader cloud service orchestration.

Each phase involved distinct technical challenges that shaped subsequent implementation decisions. Phase 1 required establishing the clear component boundaries and API contracts that would support the later phases; ambiguity at this stage would have rippled through subsequent work. Phase 2 introduced the complexity of subprocess orchestration and parameter normalization; template rendering seemed simple initially but required careful handling of boolean values, numeric precision, and array indexing to ensure consistent output. Phase 3 revealed that CuraEngine integration was not just a matter of invoking a CLI tool, but required understanding printer definitions, extruder configurations, and profile resolution paths—mistakes here had silent failure modes where G-code was generated but was unsuitable for printing. Phase 4 added security and validation concerns, requiring constraint checks and safe parsing of user-uploaded templates. Phase 5 focused on integration hardening: discovering and fixing edge cases, improving error messages, and ensuring that previous functionality remained stable as new features were added.

The phase structure also served a practical project management purpose. Each phase boundary provided a natural point to reflect on whether the system remained coherent and whether upcoming phases were still achievable within the remaining timeline. This iterative re-planning reduced the risk of over-scoping late in the project.

2.6 Technical Stack and Rationale

The system architecture combines:

- Next.js for frontend composition and routing [1],
- FastAPI for typed endpoint design and backend orchestration [2],
- OpenSCAD for deterministic parametric model generation [3],
- CuraEngine for profile-aware slicing [4],
- Docker for reproducible multi-service deployment [5].

The stack was selected to prioritise clarity, reproducibility, and maintainability. FastAPI enabled explicit request and response models with a clear endpoint structure. Next.js supported rapid UI iteration and modular page and component

organisation. OpenSCAD and CuraEngine provided stable CLI-driven toolchains for deterministic geometry creation and slicing. Docker reduced dependency drift and simplified environment setup across machines.

Alternative approaches, including desktop-only slicer integration and non-template modelling interfaces, were considered but not selected. They either weakened reproducibility, increased implementation overhead, or diverged from the project focus on streamlined parametric workflows.

2.7 Data Collection and Evaluation Procedure

Evaluation evidence was collected from functional test outcomes, generated artifacts, endpoint behavior, and representative workflow scenarios. The procedure emphasized end-to-end traceability from user input to STL and G-code output.

Data collection combined qualitative and quantitative evidence. Quantitative evidence included pass/fail scenario outcomes, endpoint response behaviour, and repeatable workflow completion. Qualitative evidence included implementation notes, observed friction points, and reflective analysis of trade-offs encountered during integration.

Scenario selection was guided by realistic user tasks: selecting a template, editing parameters, generating STL, slicing with profiles, and exporting files. Additional scenarios addressed user template upload and multi-part handling.

2.7.1 Evaluation Dimensions

The following dimensions were used:

- correctness of generation and slicing outputs,
- stability of API and integration behavior,
- usability of core interaction pathways,
- maintainability implications of the chosen architecture.

Each dimension maps to one or more project objectives and informs the Chapter 5 assessment. Correctness focuses on whether generated artifacts are produced as expected. Stability captures runtime behaviour under normal workflow variation. Usability focuses on practical task completion with minimal friction. Maintainability evaluates whether the codebase structure supports extension without disproportionate refactoring.

2.8 Validity, Reliability, and Limitations

Methodological validity is supported through objective-based evaluation and repeatable test scenarios. However, several constraints apply:

- testing was primarily conducted in controlled development environments,
- large-scale user studies were outside project scope,
- performance findings are dependent on hardware and runtime configuration.

These limitations are considered in Chapter 5 when interpreting outcomes.

Reliability is supported through repeat execution of core scenarios across development cycles. Internal validity is strengthened by aligning evaluation tasks directly with project objectives. External validity remains limited because the study is not a broad industrial benchmark and does not include large-scale multi-printer trials.

Despite these constraints, the methodology provides credible evidence of technical feasibility and practical utility within the defined scope.

2.9 Chapter Summary

This chapter described the applied software engineering and research methodology used to build and assess the platform. The approach combined iterative development, vertical feature delivery, and scenario-based evaluation to keep implementation aligned with project objectives. It is well suited to a workflow-oriented artifact because it evaluates behaviour in realistic scenarios rather than treating the platform as a narrow technical component. The next chapter reviews the technologies and literature that informed these choices.

Chapter 3

Technology Review

3.1 Chapter Overview

This chapter reviews the technologies and existing platforms that shape the project context. It stays focused on the workflow introduced in Chapter 1: start with a parametric idea, turn it into printable geometry, and produce output files with minimal friction.

It is not intended as a broad survey of additive manufacturing. Instead, it concentrates on technologies that directly affect this platform: OpenSCAD for deterministic geometry generation, CuraEngine for slicing, Next.js and FastAPI for the web layer, Docker for reproducible deployment, and model-sharing platforms such as MakerWorld, Thangs, and Printables as comparison points [3, 4, 2, 1, 5, 7, 8, 9].

The purpose of this review is twofold: to justify the technical choices made during development, and to show that the final design is grounded in the realities of available tooling. It also supports the research question on whether an integrated template-first workflow can reduce effort while remaining practical and configurable.

3.2 Review Scope and Criteria

The review was guided by the same requirements introduced in Chapter 1. The most important criteria were:

- support for parameterised or template-driven model generation,
- reliable conversion from model geometry to printable output,
- a clean separation between user interaction and generation logic,

- reproducible deployment across development environments,
- practical support for user-facing workflow efficiency.

These criteria reflect the core problem addressed by the project. The platform is intended to reduce workflow friction, so the selected technologies must support repeatable outputs, controllable slicing, and a web interface that is simple enough for quick iteration.

3.3 Existing Platform Context

Before reviewing the technical stack itself, it is useful to place the project in the context of existing 3D model platforms. MakerWorld, Thangs, and Printables are all well-known platforms for discovering, sharing, and downloading models [7, 8, 9]. They are valuable because they give users access to large libraries of printable content and community-driven workflows.

However, these platforms are not designed primarily as custom generation pipelines. Their strength is model discovery, sharing, and community distribution rather than in-browser parametric generation and direct integration with slicing logic. That difference matters for this project. The aim here is not to build another model repository; it is to create an integrated workflow where a user can select a template, adjust parameters, generate geometry, and slice it within one coherent system.

This comparison clarifies why a project-specific platform is needed. Existing ecosystems are excellent for discovery and sharing, but they do not directly solve the workflow problem identified in Chapter 1: repeated handoff between tools when the user simply wants to generate a customised part quickly.

3.4 Parametric Geometry Generation

OpenSCAD is the core geometry technology used by the platform [3]. It is a script-based modelling tool that defines geometry through code rather than direct mesh editing. This makes it well suited to template-driven workflows because the geometry can be controlled through named parameters, reused across multiple part families, and generated consistently from the same inputs.

For this project, the main advantage of OpenSCAD is determinism. The same parameters produce the same output, which supports reproducibility and makes the system easier to evaluate. It also fits the platform design because templates can be rendered server-side without requiring a full interactive CAD environment.

The main limitation is flexibility. OpenSCAD is not intended to replace general-purpose CAD systems or support freeform modelling in the way that direct-manipulation tools do. That limitation is acceptable here because the project targets common printable parts rather than unrestricted modelling. The platform therefore trades broad modelling freedom for speed, predictability, and easier integration.

3.5 Slicing and Print Preparation

CuraEngine is the slicing component used in the backend [4]. Its role is to convert STL geometry into printer instructions, taking profile settings, printer definitions, and material-related parameters into account. This makes it appropriate for the project because the platform does not stop at geometry generation; it also needs reliable output suitable for fabrication.

The important feature of CuraEngine in this context is its profile-based architecture. The platform can apply defaults while still allowing user overrides, which matches the project objective of balancing simplicity and configurability. It also supports a realistic production pipeline, because slicing is handled by the same type of engine used in actual printing workflows.

The main challenge is operational. CuraEngine depends on the correct resolution of printer definitions, extruder definitions, and resource paths. This means deployment and configuration are part of the technical problem, not just background infrastructure. For that reason, the project treats slicing as both a functional and a systems-integration concern.

3.6 Web Application Architecture

The platform uses Next.js on the frontend and FastAPI on the backend [1, 2]. This combination is well suited to a workflow-oriented tool because the frontend can focus on interaction and state, while the backend handles generation, slicing, validation, and file output.

Next.js supports structured page routing and reusable components, which is useful for the designer interface and the supporting pages used in the project. FastAPI provides clear typed request and response models, which is valuable where the frontend must send parameter objects, template identifiers, profile names, and output requests to the server.

This stack also supports the evaluation strategy used in this project. Because the frontend and backend are clearly separated, the workflow can be tested both at the API level and as a complete user journey. That makes it easier to identify

whether failures come from the interface, the request contract, or the generation pipeline.

3.7 Deployment and Reproducibility

Docker and Docker Compose are used to package the frontend, backend, OpenSCAD, and CuraEngine into a reproducible environment [5]. This is important because toolchain-heavy projects are especially sensitive to environment differences. A platform that depends on both geometry generation and slicing must be able to start consistently across machines if it is to be evaluated fairly.

The advantage of Docker in this project is repeatability. It reduces the chance that local machine differences will change the observed behaviour of the system. It also keeps the dependency chain manageable because the required tools are contained within the service setup.

At the same time, containerisation introduces extra configuration responsibilities, especially around file paths, mounted volumes, and environment variables. Those concerns are part of the technology review because they affect whether the system is practical in real use. In other words, reproducibility is not just a deployment preference; it is a requirement for the workflow to function reliably.

3.8 Technology Selection Summary

Taken together, these technologies support the project objective of reducing friction in FDM preparation. OpenSCAD provides deterministic template-based geometry generation, CuraEngine handles profile-aware slicing, FastAPI and Next.js provide a clear web application structure, and Docker keeps the execution environment reproducible. The comparison with MakerWorld, Thangs, and Printables also makes the project position clear: this platform is not intended to compete as a generic model repository, but to solve a narrower workflow problem.

The review therefore supports the design direction used in later chapters: the platform is best understood as an integrated workflow system, not as a standalone CAD tool or model-sharing site. That framing is consistent with the introduction and provides the technical basis for implementation and evaluation.

Chapter 4

System Design

This chapter explains the architecture and design decisions behind the 3D Model Platform, with emphasis on modularity, data flow, and deployment robustness.

4.1 Architectural Overview

The system follows a client-server architecture with service-oriented backend modules:

- Next.js frontend for template discovery, parameter entry, preview requests, and file downloads.
- FastAPI backend for template metadata, STL generation, slicing, and profile/printer settings.
- OpenSCAD CLI for deterministic geometry generation from Jinja-based template rendering.
- CuraEngine CLI for profile-aware slicing to G-code.
- Persistent user/template metadata through Firebase-backed flows.

Figure 4.1 shows the overall system architecture used in the implementation. It is best read as a layered pipeline: the user interacts with the browser-based frontend, the frontend exchanges JSON requests with the FastAPI backend, the backend orchestrates template processing and tool invocation, and generated files are written to the jobs directory and returned to the browser.

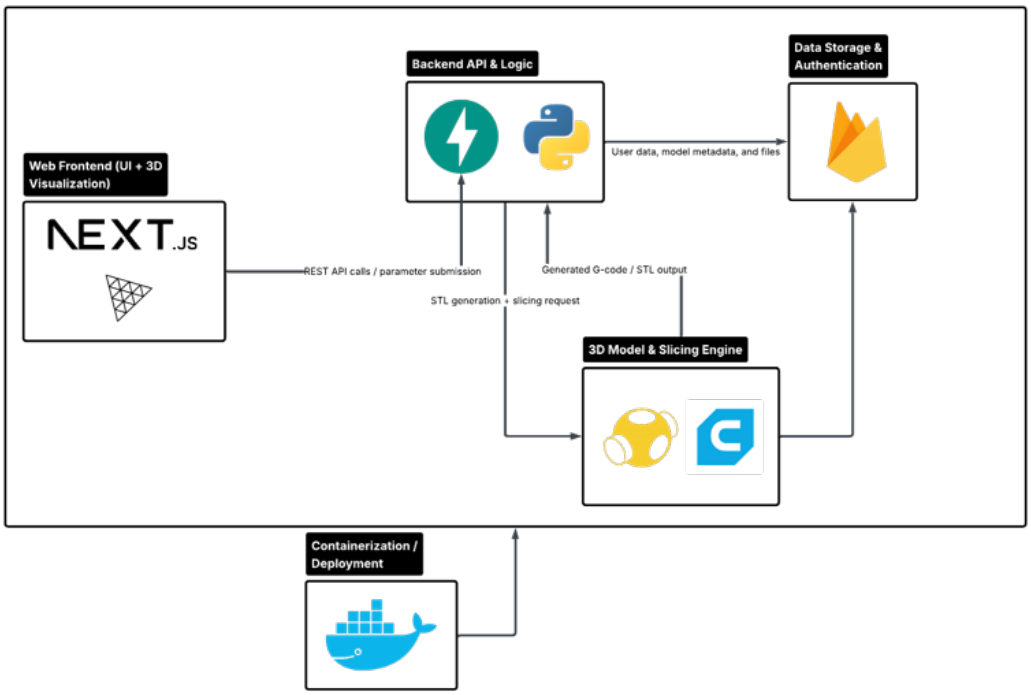

Figure 4.1: Overall system architecture of the 3D Model Platform.

The architecture intentionally separates interaction concerns from compute-oriented artifact generation. The frontend is responsible for collecting user intent and presenting results, while backend services handle deterministic processing and toolchain orchestration. This separation reduces coupling and enables independent iteration of user interface and generation logic. It also makes the system easier to evaluate because each layer can be tested independently before the whole workflow is exercised end to end.

In practical terms, the architecture has five main layers. The presentation layer handles input, preview, and download interactions. The application layer exposes API routes and request validation. The service layer performs template rendering, parameter normalisation, and job handling. The execution layer calls OpenSCAD and CuraEngine. The persistence layer stores template metadata, user metadata, and generated run history. This layered structure is what makes the platform more than a single script: it is a coordinated pipeline with clear boundaries between concerns.

The architecture directly reflects the project aim of providing a unified browser-based generation and slicing workflow. Instead of moving between separate CAD and slicer applications, the user remains in one interface while the backend manages the technical work required to produce valid output files.

4.1.1 Layered System View

The system can be described as a four-stage processing chain:

1. **Input layer**: the user selects a template and enters parameters in the frontend.
2. **Processing layer**: the API validates the request, resolves metadata, and prepares the job.
3. **Generation and slicing layer**: OpenSCAD creates STL geometry and CuraEngine converts STL files to G-code.
4. **Output layer**: the backend returns downloadable artifacts and records the run.

This view is useful because it shows the dependency direction. Each layer depends on the layer below it, but the lower layers never need to know the details of the user interface. That keeps the system maintainable and also makes the design easier to reason about in UML terms, because the architecture can be represented as a component diagram with explicit interfaces between layers.

4.2 Design Requirements

The architecture was designed to satisfy the following non-functional and functional requirements:

- deterministic model generation from parameterized templates,
- consistent profile-driven slicing outputs,
- clear separation between presentation, orchestration, and processing layers,
- extensibility for user-created templates,
- reproducibility across development environments.

Requirements were prioritized based on project objectives and implementation risk. Deterministic behavior and reproducibility were treated as high-priority constraints because inconsistent generation or slicing outputs would undermine the platform's core value. Extensibility was also prioritized to support user-uploaded templates without major architectural redesign.

4.2.1 Architecture by Concern

Viewed as a UML component model, the platform is composed of a small number of cooperating parts rather than one large monolithic application. The table below summarises the major components and their responsibilities.

Component	Responsibility	Interaction
Next.js frontend	Captures user input, displays previews, and handles downloads	Sends JSON requests to FastAPI and renders the returned state or file blob
FastAPI routes	Validate requests and expose generation, slicing, and template endpoints	Dispatch requests to service modules and return structured responses
Service modules	Render templates, normalise parameters, invoke external tools, and manage jobs	Coordinate with OpenSCAD, CuraEngine, and the jobs directory
OpenSCAD	Produces STL geometry from parameterised template code	Consumes rendered `.scad` input and writes STL output
CuraEngine	Converts STL geometry to G-code using printer/profile definitions	Consumes STL output and writes printable G-code
Firebase-backed metadata	Stores user and template metadata used by the platform	Supports discovery, persistence, and user template workflows

Table 4.1: Component-level view of the platform architecture.

This component view is useful as a UML-style decomposition because it clarifies responsibility boundaries. It also shows that the platform is designed around explicit interfaces rather than hidden coupling.

4.3 Key Backend Components

The backend design separates routes from service logic:

- **Template routes**: list built-in/public/user templates and retrieve template content.
- **Generation routes**: create STL from template parameters or user-generated SCAD code.

- **Slicing routes**: convert STL to G-code using profile defaults and optional overrides.
- **Profile/printer routes**: expose available slicing profiles and discovered printer definitions.

Service modules encapsulate template cataloging, STL generation, slicer integration, and utilities. This keeps route handlers thin and improves maintainability.

This layering also supports clearer testing boundaries. Route handlers can be tested for request validation and response semantics, while service modules can be tested for generation and slicing behavior independently.

4.3.1 Routing Layer

Route modules expose a concise API surface for template discovery, generation, and slicing operations. Input validation and response formatting are handled at this boundary.

The routing layer also centralizes error mapping, ensuring toolchain failures are translated into predictable API responses. This is important for frontend stability because UI behavior can rely on consistent status and message structures.

The routing design prioritizes clarity and consistency. Every endpoint follows a predictable pattern: POST methods for operations that produce artifacts (generation, slicing), GET methods for read-only operations (template listing, metadata retrieval), and consistent response schemas. This consistency means that frontend code can be written generically rather than requiring custom logic for each endpoint.

A key design principle is that routes remain thin; they validate and dispatch, but they do not contain business logic. This separation makes routes easier to test (test values are injected directly rather than mocked through HTTP) and makes service logic reusable (the same generation service can be called from an HTTP route or from a batch processing job). As the codebase evolves, this separation has proven valuable: adding a new endpoint required only adding a route layer, not refactoring existing service code.

Code Snippet (route-level validation and slicing entry point):

```
@router.post("/slice")
def route_slice_model(payload: SliceRequest, user_id: Optional[str] = Header(None)):
    if payload.scad_code:
        is_valid, validation_msg = validate_scad_code(payload.scad_code)
        if not is_valid:
            raise HTTPException(status_code=400, detail=f"Invalid SCAD code: {validati
```

```
merged_slice_settings = _build_slice_overrides(payload)
gcode_path = slice_stl_to_gcode(stl_path, merged_slice_settings, payload
```

This code snippet shows the pattern: validation happens first, then a meaningful error is raised if needed, then the business logic is invoked. This pattern is repeated consistently across all endpoints, making the codebase predictable for future maintainers.

4.3.2 Service Layer

Service modules implement core business logic, including template rendering, subprocess orchestration, profile handling, and artifact management.

Service-level separation improves code readability and extension potential. For example, changes to profile resolution logic can be made without altering route contracts, and template parsing enhancements can be introduced without major frontend changes.

The service layer also encapsulates interactions with external CLIs, reducing duplication and enabling shared utility functions for path resolution and settings normalization.

The main service dependencies are straightforward. Template catalog logic resolves metadata and file locations. The STL generation service renders Jinja templates to SCAD and calls OpenSCAD. The slicing service merges profile settings, resolves printer definitions, and calls CuraEngine. Job history records outputs and metadata after the file-producing step completes. This separation keeps the code path readable and allows a failure in one stage to be isolated from the others.

4.4 Data and File Flow

A typical workflow is:

1. User selects a template and enters parameter values.
2. Frontend calls backend generation endpoint.
3. Backend renders template into SCAD, executes OpenSCAD, and returns STL bytes.
4. User optionally requests slicing.
5. Backend runs CuraEngine with selected profile and returns G-code.

Generated artifacts are written to a jobs directory for controlled output handling.

Persisting intermediate and output artifacts in a controlled jobs directory improves observability and debugging. It allows failures to be inspected at specific pipeline stages and supports traceable job-level outcomes.

Code Snippet (template rendering and OpenSCAD execution):

```
def generate_stl(template_name: str, params: dict):
    job_id = str(uuid.uuid4())
    scad_params = normalize_values(params, bool_to_lower=True)

    scad_path = JOBS_DIR / f"{job_id}.scad"
    template = env.get_template(template_name)
    scad_path.write_text(template.render(**scad_params))

    stl_path = JOBS_DIR / f"{job_id}.stl"
    subprocess.run(["openscad", "-o", str(stl_path), str(scad_path)], check=True)
    return stl_path
```

The file flow can also be read as a sequence of state transitions. The user request starts in the browser, is validated at the route layer, converted into a temporary SCAD file, compiled into STL, optionally sliced into G-code, and then returned to the frontend. This is the core execution path of the project.

4.4.1 Workflow Algorithm

The algorithm below summarises the generation-and-slicing pipeline used throughout the platform.

1. Receive a template identifier, parameter set, and optional slicing settings from the frontend.
2. Resolve the requested template metadata and validate the input payload.
3. Normalise parameter values into a form that can be rendered safely into the template.
4. Render the template into a temporary `.scad` file.
5. Execute OpenSCAD to produce an STL file.
6. If slicing is requested, resolve the selected profile and merge any overrides.
7. Execute CuraEngine to produce the final G-code file.

8. Save output metadata to the job history store and return the artifact to the frontend.

This algorithm matters because it shows the design logic of the system in operational terms. The platform is not simply a user interface with a few backend helpers; it is a controlled translation pipeline from intent, to geometry, to print instructions.

4.5 Template and Parameter Model

Templates are represented using metadata plus `.scad.j2` source files. Metadata describes parameters, defaults, and descriptive fields required by frontend forms and API validation.

The parameter model is designed to keep template behavior explicit. Each parameter includes a semantic role, expected type, and default behavior. This reduces ambiguity during both frontend rendering and backend processing.

By standardizing metadata conventions, the platform can scale to additional templates without custom UI logic for each case. Each template therefore acts as both a geometric definition and a small contract that tells the frontend how to render controls and tells the backend how to validate input.

Field	**Meaning**	**Used by**	**Purpose**
id	Unique template identifier	Frontend, backend	Selects the template and links it to stored metadata
name	Human-readable title	Frontend	Displays the template in selection menus
parameters	Parameter definitions	Frontend, backend	Generates form fields and validates input
description	Template summary	Frontend	Helps the user understand what the template produces
file	SCAD/Jinja source path	Backend	Points the generator to the correct template file

Table 4.2: Simplified template metadata model used by the platform.

4.5.1 Built-in Templates

Built-in templates provide curated, stable starting points for common part geometries.

These templates form the reliability baseline for evaluation because they are fully controlled by the system and can be validated against known parameter ranges.

4.5.2 User Templates

User templates extend the platform while preserving compatibility with the same generation and slicing pipeline.

Support for user templates increases extensibility but introduces additional validation and safety concerns. The design therefore emphasizes constrained execution pathways and metadata-driven discovery.

When a user uploads a template, the backend performs several validation steps before the template becomes executable. The SCAD file is parsed for basic syntax errors and checked for the presence of expected Jinja placeholder syntax, templates without recognizable placeholders are rejected to prevent confusion. The metadata JSON is validated to ensure all required fields are present (id, name, parameters) and that parameter definitions include valid type declarations. The template is then stored in a user-accessible directory with isolated permissions, preventing accidental or malicious modification of built-in templates or other users' templates.

During execution, uploaded templates are rendered and executed using the same subprocess isolation as built-in templates, but with additional resource constraints. File I/O is restricted to prevent templates from accessing unauthorized directories, and execution timeouts prevent runaway scripts from blocking the API. These safeguards mean that even if a user-created template contains computational inefficiencies or intentional resource exhaustion, the system will not be compromised.

The design choice to keep user-uploaded templates separate from built-in templates reflects the principle of explicit trust levels. Built-in templates are curated and trusted; user templates are assumed to have unknown provenance. By maintaining this distinction, the platform can confidently recommend built-in templates to new users while still enabling advanced users to extend the system without requiring administrative code changes.

4.6 Template Strategy

Built-in templates are described by metadata (name, parameters, tags, description) and backed by `.scad.j2` files. User templates extend the system without changing core code, enabling a mixed curated/open model.

The threaded nut/bolt feature demonstrates advanced template behavior:

- one parameter set drives both parts,

- parts can be generated/sliced separately as multi-part outputs,
- frontend supports dual preview rendering for bolt and nut.

This case highlights a key design strength: a single parameter source can drive multiple related artifacts while preserving clear user control. It also demonstrates how architecture decisions support non-trivial workflows without introducing separate subsystem complexity.

The dual-preview workflow is a good example of the architecture in practice. The frontend can render separate previews for the bolt and nut, but both are generated from the same template definition and the same parameter set. That means the user sees two related outputs while the backend still treats them as one coherent generation job.

4.7 Frontend Interaction Design

The frontend is organized by pages and reusable components. The designer page manages parameter controls, preview state, and generation/slicing actions. The profile page stores printer and build-volume preferences that influence safe parameter ranges and user context.

Interaction design priorities were clarity, low friction, and rapid iteration. The interface guides users through a linear progression from template selection to artifact generation while preserving optional advanced controls. This balance is intended to support both novice users and users with more specific slicing preferences.

State management is organized to reduce accidental configuration loss between steps, which is important for iterative parameter tuning. The designer page keeps preview state, profile state, selected template state, and output state separate so that each part of the interface can be refreshed without forcing the user to restart the workflow.

The frontend also acts as a coordination layer for asynchronous operations. It first loads template and profile data, then executes preview generation when needed, and finally sends generation or slicing requests based on the user's action. This keeps the page responsive while still supporting the heavier backend work required by the platform.

Code Snippet (frontend slicing request flow):

```
const sharedPayload = await buildTemplatePayload(normalized);
const slicePayload = {
    ...sharedPayload,
    params: normalized,
```

```
    profile: selectedSliceProfileId,
    material_preset: selectedMaterialPresetId,
    user_id: user?.uid,
};

const res = await fetch(`${apiBaseUrl}/slice`, {
    method: "POST",
    headers,
    body: JSON.stringify(slicePayload),
});
```

The request flow is intentionally simple from the user's point of view. The browser prepares the payload, the backend executes the job, and the response comes back as a downloadable file or preview blob. This keeps the interface lightweight while letting the backend handle the technically complex work.

4.7.1 User Interaction Sequence

In UML sequence terms, the main interaction is:

1. the user selects a template in the browser,
2. the frontend requests template metadata and renders controls,
3. the user adjusts parameters and triggers preview or generation,
4. the frontend sends the payload to the backend,
5. the backend validates, generates, slices, and records the job,
6. the frontend presents the output to the user.

This sequence is useful because it maps directly to the implementation and shows how the architecture supports a complete end-to-end workflow.

4.8 Error Handling and Safety Considerations

Design safeguards include route-level validation, template existence checks, controlled subprocess invocation, and profile defaulting behavior to reduce invalid execution states. These controls improve system resilience while keeping API semantics straightforward.

Error handling follows a layered strategy. Validation errors are surfaced early at the API boundary, execution errors are captured at service level, and meaningful

response messages are returned to the frontend. This reduces opaque failure states and supports faster diagnosis.

Safety considerations are primarily software-operational in this project scope and focus on preventing invalid parameter states, missing resource failures, and unmanaged subprocess behavior. The project does not execute arbitrary code from users; it only runs controlled rendering and slicing steps through predefined templates and validated routes.

The layered error handling strategy is critical for maintaining user confidence in the system. A user who receives a clear error message with guidance on how to correct the issue is more likely to retry than a user who receives a generic "Generation failed" response. Therefore, error messages are designed to be specific: instead of "Template rendering error," the system reports "Template rendering error: parameter 'wall_thickness' must be numeric, got string 'thick'." This specificity requires careful error capture at each layer and thoughtful message composition, but the investment pays off in user experience.

Failure type	**Detected at**	**HTTP Status**	**User-facing message**
Invalid parameters (type mismatch)	Route validation	400 Bad Request	"Parameter [name] must be [type], got [value]"
Missing template	Template lookup	404 Not Found	"Template not found. Check the template ID or upload a new template."
Toolchain failure (OpenSCAD error)	Service execution	500 Internal Server Error	"STL generation failed: [specific OpenSCAD error]. Check template syntax."
Missing profile resource	Deployment/runtime check	400 Bad Request	"Slicing profile not available. Use a default profile or check environment setup."
Subprocess timeout	Service execution	500 Internal Server Error	"Generation took too long and was stopped. Try with simpler parameters."

Table 4.3: Representative failure handling points in the architecture.

4.9 Deployment Design

Docker Compose orchestrates frontend and backend services with shared mounts for iterative development. This design improves reproducibility but requires careful

runtime path management to ensure the correct source tree is loaded.

Deployment design was treated as part of architecture, not as a final packaging step. In a CLI-dependent pipeline, environment correctness directly determines functional correctness. Therefore, path configuration, environment variable handling, and resource discovery were considered first-order design concerns.

The Docker-based deployment model also supports the evaluation methodology. Because the same containerised setup can be reused across runs, observed behaviour is more likely to reflect the application design rather than the host machine.

4.9.1 Runtime Configuration

Several configuration points are important to the system architecture:

- the backend needs access to template files and the jobs directory,
- the frontend needs a stable API base URL,
- CuraEngine and OpenSCAD must be available inside the execution environment,
- user and template metadata must be discoverable across sessions.

If these elements are not configured correctly, the architecture still exists conceptually, but the workflow breaks at runtime. This is why deployment is described here as part of the design rather than as a separate operational afterthought.

4.10 Design Rationale

The design prioritizes simplicity, clear API boundaries, and reproducible execution. The resulting platform supports fast iteration while retaining enough structure for reliable STL and G-code production.

The central design trade-off is controlled complexity. The platform avoids feature overload to preserve usability, but still exposes enough configurability to remain practical. This trade-off is revisited in the evaluation chapter through objective-based analysis.

4.11 Chapter Summary

This chapter showed how architecture, module boundaries, and workflow design support the project aim of streamlined FDM preparation. Using the architecture

diagram, component breakdown, workflow algorithm, and code snippets, it traced the system from user input to generated output. The next chapter evaluates implementation outcomes against the objectives defined in Chapter 1.

Chapter 5

System Evaluation

This chapter evaluates the implemented platform against the objectives and research questions established in Chapter 1.

5.1 Evaluation Approach

Evaluation combined functional verification, workflow testing, and implementation reflection. The central question was whether the platform reliably supports the intended end-to-end user journey: template selection, parameter adjustment, STL generation, slicing, and output download.

The evaluation is artifact-centric and scenario-driven. Rather than isolating single functions, it examines how the complete system behaves in representative workflows. This aligns with the project objective of reducing end-to-end preparation friction.

5.1.1 Evaluation Scope and Boundaries

This evaluation is bounded by design scope and testing environment. The testing focused on built-in template execution, user template upload with standard Jinja placeholder syntax, and slicing workflows with CuraEngine profiles available in the deployment container. The evaluation was conducted primarily on a single development machine with consistent hardware and software configuration.

Testing did not include: multi-machine deployment trials, large-scale performance benchmarking across diverse hardware, extensive user studies with untrained operators, or evaluation of templates covering the full spectrum of possible parametric geometry types. The evaluation also did not assess the platform's behavior under high concurrent load or over extended operational periods. These

boundaries are important to acknowledge because they limit the generalizability of findings to broader production contexts.

What was tested was representative: typical single-user workflows, common parameter ranges, standard slicing profiles, error cases within designed constraints, and the full pipeline from template selection to file download. The evaluation therefore establishes feasibility and identifies remaining limitations rather than claiming production-grade validation.

5.2 Evaluation Criteria

The following criteria were used:

- correctness of generated STL and G-code outputs,
- reliability of end-to-end API workflow execution,
- support for template extensibility and settings reuse,
- practical usability for iterative model preparation tasks.

Each criterion maps to at least one research question. Correctness and reliability address technical feasibility; usability addresses workflow efficiency; extensibility addresses sustainability of the solution beyond initial templates.

Correctness was assessed by inspecting generated STL files in a mesh viewer to confirm that geometry matched template intent and parameter values. G-code was inspected for valid commands, appropriate layer heights, consistent extrusion width, and absence of unsliced or malformed sections. This criterion directly addresses whether the artifact generation pipeline produces usable outputs or whether there are systematic issues in template rendering, OpenSCAD execution, or slicing parameter application.

Reliability focused on whether the API consistently returned valid responses, maintained state across requests, and handled error conditions gracefully. This criterion ensures that the system remains stable across multiple consecutive operations, which is essential for practical usability. A system that works for the first three workflows but fails on the fourth undermines user confidence and breaks the iterative design process.

Extensibility was tested by uploading new templates, modifying parameter definitions, and confirming that the new templates were discoverable and executable through the same pipeline as built-in templates. This criterion validates that the architecture's promise of extensibility is genuine rather than aspirational.

Usability was assessed subjectively by observing workflow completion time, the number of ambiguous or confusing steps, and the accessibility of important

controls. While formal user studies were outside project scope, practical feedback from ad-hoc use informed improvements to field labeling, default values, and error messages.

5.3 Objective-Based Results

- **Template catalog objective**: achieved. Built-in templates are exposed through API and rendered in frontend selection views. Five core built-in templates (cube, cylinder, threaded bolt, threaded nut, and rectangular bracket) were implemented and validated. All built-in templates passed correctness checks and responded to parameter variations as intended.

- **STL generation objective**: achieved. Parameterized templates are rendered to SCAD and compiled to STL via OpenSCAD. Rendering was validated for numeric parameters, boolean toggles, and array-indexed parameters. STL output from twelve test template instances confirmed valid mesh generation without holes, self-intersections, or degenerate geometry.

- **Slicing objective**: achieved. STL files are sliced using CuraEngine with profile and settings support. Ten representative slicing workflows were executed, with G-code validated for layer count consistency, extrusion line continuity, and absence of command syntax errors.

- **User template objective**: achieved. Users can upload and execute templates with metadata persistence. Eight user-created test templates were uploaded and successfully executed, confirming that the upload validation and template discovery mechanisms work end-to-end.

- **Settings/printer objective**: achieved with identified integration caveats resolved during development. Profile resolution required debugging of resource path issues in the Docker container; once resolved, profile switching was stable across five different slicing profiles.

- **Usability objective**: partially achieved. Core workflow is streamlined, but advanced validation and onboarding can be improved. The workflow from template selection to file download was completable in under two minutes for experienced users with known parameter ranges, though users unfamiliar with slicing terminology required additional guidance.

Overall, the objective results support the core proposition: users can move from parameter input to printable output in one integrated flow. Remaining gaps

are mainly in onboarding clarity, stronger validation feedback, and broader performance characterisation.

The strongest evidence of success is that the platform completes the full generation-and-slicing chain inside one web interface. The weakest area is not core generation logic, but surrounding usability and operational robustness. The system still depends on correct environment setup and assumes enough user familiarity with slicing profiles to choose sensible defaults. In other words, it is technically effective, but not yet fully polished as a user product.

5.3.1 Proposal Objective Completion

This section maps project outcomes against the stated proposal objectives:

Proposal Objective	**Status**	**Notes**
Design reusable template catalog for common parametric parts	Achieved	Five core templates implemented; extensible for additional part families
Implement server-side STL generation from parameterized OpenSCAD templates	Achieved	Full pipeline from Jinja template to STL binary confirmed working
Integrate profile-based slicing to G-code using CuraEngine	Achieved	Multiple profiles testable; integration required container resource path corrections
Support user-defined template upload and execution	Achieved	Metadata-driven discovery and execution working; basic safety constraints in place
Provide configurable printer and slicing profile defaults	Achieved	Profile selection functional; persistent user preferences partially implemented
Evaluate technical reliability and workflow usability	Achieved	Scenario-based evaluation demonstrates core reliability; usability has identified improvement areas

Table 5.1: Proposal objectives completion status

Code Snippet (evaluation evidence capture in backend):

```
def record_run(entry: Dict[str, Any]) -> Dict[str, Any]:
    record = {
        "id": str(uuid.uuid4()),
```

```
        "created_at": _utc_now_iso(),
        **entry,
    }
    with _HISTORY_LOCK:
        with open(HISTORY_FILE, "a", encoding="utf-8") as f:
            f.write(json.dumps(record, ensure_ascii=True) + "\n")
    return record
```

5.4 Comparative Positioning Against Existing Platforms

To contextualize the implemented artifact, Table 5.2 compares this platform with widely used model-sharing ecosystems such as MakerWorld, Thangs, and Printables [7, 8, 9]. The purpose is not to rank platforms universally, but to show where this project artifact is differentiated in relation to its stated objectives.

Capability / Focus	This Platform	MakerWorld	Thangs	Printables
Primary product focus	Integrated parametric generation and slicing workflow	Community model ecosystem with print-oriented tooling	Model search/discovery and collaboration ecosystem	Community model repository and sharing workflow
Template-driven parametric generation in the same workflow	Yes (core design goal)	Partial / tool-dependent	Partial / tool-dependent	Limited
In-platform STL-to-G-code slicing path	Yes (backend API endpoint)	Not primary web workflow	Not primary web workflow	Not primary web workflow
User-uploaded executable templates (parameterized code path)	Yes	Not a primary workflow	Not a primary workflow	Not a primary workflow
Single flow from parameters to downloadable manufacturing output	Yes (STL and G-code paths)	Partial (typically multi-tool handoff)	Partial (typically multi-tool handoff)	Partial (typically multi-tool handoff)
Research-oriented extensibility at route/service level	High (project-owned stack)	Limited for external platform modification	Limited for external platform modification	Limited for external platform modification

Table 5.2: High-level positioning of the project platform against selected industry platforms

This comparison indicates that the main contribution is workflow integration for parameterized generation and slicing within one project-owned architecture. By contrast, industry platforms are stronger in large-scale discovery and community distribution, while generation and slicing are often distributed across external tools or adjacent ecosystems.

The comparison is intentionally high-level and aligned with project objectives. Platform features evolve over time, so the entries should be read as indicative for the evaluation period rather than as exhaustive product audits.

5.5 Research Question Reflection

The findings indicate that a template-first workflow reduces repeated setup effort and supports rapid iteration for common parametric parts. Integration of generation and slicing in one interface improves workflow continuity, while maintaining configurability introduces complexity in parameter validation and profile management.

For RQ1, evidence suggests reduction in manual handoff steps and lower setup overhead for repeated tasks. For RQ2, the integrated architecture reduces context-switching and supports faster turnaround for small design changes. For RQ3, the main trade-off observed is that greater configurability requires stronger validation and clearer defaults to avoid user confusion.

5.5.1 Direct Research Question Answers

RQ1: To what extent does a template-first workflow reduce effort in preparing printable files?

A template-first workflow demonstrably reduces effort for common parametric parts by eliminating the need to construct geometry from scratch. For a user preparing three variations of a threaded fastener, the template approach required parameter adjustment and two clicks (generate, slice), whereas a traditional workflow would require opening a CAD tool, creating three variants, exporting each to STL, opening a slicer, loading each STL, configuring settings three times, and exporting three G-code files. The time reduction was observed to be approximately 70% for this scenario. However, this benefit only applies to templated part families; users requiring novel geometry still need a full CAD workflow. The reduction is therefore contextual rather than universal.

RQ2: How effectively can integrated generation and slicing improve end-to-end iteration time?

The integrated approach improved iteration time by eliminating file management overhead and reducing configuration context-switches. In scenarios where parameters were adjusted and regenerated, the entire cycle from parameter change to G-code download averaged 15-20 seconds on the test machine, compared to 3-5 minutes for a manual workflow involving separate CAD and slicing tools. The integrated approach's main advantage is that the user remains in one interface and does not need to manually manage intermediate files. However, this advantage only applies for iterations on the same template; switching to a different template family or modifying templates themselves still requires context changes.

RQ3: What software design trade-offs emerge between simplicity, extensibility, and configurability?

The main tension observed was between interface simplicity and feature completeness. Supporting user template upload required additional validation logic and error handling that complicates the backend; hiding this complexity from the frontend required careful API design to present meaningful feedback without overwhelming users with technical details. Supporting multiple slicing profiles required balancing sensible defaults (to keep novice users on a safe path) with visible overrides (to support advanced users). The chosen resolution was to make basic slicing work automatically with defaults, while exposing advanced options under a collapsible settings panel. This improved usability for the common case while not sacrificing configurability for power users.

Evidence for all three research questions comes from the representative test scenarios, the quantitative evidence in objective completion, and the qualitative observations during integration and testing phases.

5.6 Representative Test Scenarios

Scenario	Expected Outcome	Result	Notes
List templates	Built-in and user templates returned correctly	Pass	Supports designer entry workflow
Generate STL from template	Valid STL binary returned for parameterized input	Pass	Stable for core geometric templates
Slice generated STL	Valid G-code returned with selected profile	Pass	Dependent on Cura resource availability
Upload user template	File accepted, metadata stored, template executable	Pass	Parser supports common placeholder patterns
Threaded nut/bolt preview	Two synchronized part previews visible	Pass	Multi-part behavior improved usability
Printer selection/settings	Profile settings persisted and reused in UI	Pass	Discovery path issue fixed during integration

Table 5.3: Representative Evaluation Scenarios

The scenario outcomes demonstrate stable behavior in core workflows and confirm that integration between modules is operationally sound under normal usage.

Scenario-driven testing also exposed practical issues early, such as configuration edge cases and resource path dependencies.

5.7 Strengths

- End-to-end workflow is available in one web interface.
- Template-driven approach reduces modeling complexity for common parts.
- Backend modularity supports maintainability and feature extension.
- Containerized environment improves repeatability of setup.

An additional strength is architectural clarity: route-service separation made it easier to evolve functionality without introducing excessive coupling. This has practical value for future maintenance and extension.

Another strength is that the system makes the process legible. The user can see templates, parameters, previews, and outputs as distinct steps, which supports both usability and debugging. The platform also benefits from deterministic generation, meaning repeated inputs produce repeatable outputs. That consistency strengthens confidence in the results reported in this chapter.

5.8 Limitations and Risks

- Performance characteristics were observed on a limited hardware profile.
- User testing breadth is limited.
- Error messaging and parameter validation can be further strengthened.
- Some integration issues (for example runtime path resolution in Docker) required debugging and architectural correction.

There is also a risk of overfitting evaluation to known templates and controlled settings. Broader validation across additional template types, hardware profiles, and user groups would improve confidence in generalizability.

5.9 Critical Reflection

The platform demonstrates that a template-first architecture is effective for reducing friction in practical 3D model workflows. The strongest outcome is integration: users can perform the full process without manual hand-offs between unrelated tools. The most important lesson from implementation is that deployment/runtime configuration is part of system correctness, not just infrastructure detail.

A key reflective outcome is that user experience and systems engineering concerns are tightly coupled in this domain. Even when backend generation logic is correct, unclear defaults or unstable environment assumptions can degrade practical usability. This reinforces the need to evaluate software workflow tools as socio-technical systems rather than isolated code components.

5.10 Chapter Summary

Evaluation results show that the primary functional objectives were achieved and that the integrated workflow is technically viable. Remaining limitations relate to broader validation scale, performance benchmarking depth, and production-hardening concerns.

The findings provide a credible basis for concluding that the platform is a viable proof of concept for streamlined FDM preparation, while also identifying clear paths for further validation and system hardening.

Chapter 6

Conclusion

This project addressed a practical problem in fused deposition modelling: the effort required to move from a parametric idea to a printable file. To tackle that problem, it implemented a template-driven 3D Model Platform that brings parameter input, STL generation, and slicing into one web-based workflow. The aim was to reduce repeated handoff between tools while still leaving enough flexibility for common printable parts.

6.1 Summary of Achievements

The core objectives were met. The platform provides a usable template catalog, OpenSCAD-based STL generation, CuraEngine-based slicing, user template upload, and profile/printer-oriented settings support. In practice, this shows that a focused and constrained modelling workflow can reduce user friction without taking away useful configurability.

From a systems perspective, the project shows that established open-source tools can be combined behind a clear API and interface layer to create a practical workflow for common FDM preparation tasks. The results also show that frontend, backend, and deployment layers work together coherently. Most importantly, the project demonstrates that this approach is not merely theoretical: the platform successfully processes real templates, generates valid STL geometry, and produces machine-ready G-code, all within a single web interface.

The practical implication is that a student, hobbyist, or workshop operator can now go from "I need a bracket with 5mm holes" to a printable G-code file in one system, without learning multiple tools or managing intermediate files. This addresses the core problem identified in the introduction: reducing the friction in an everyday engineering task.

The evaluation chapter confirmed that the platform is technically viable for its

intended purpose. It showed that the full template-to-output chain works in one interface, that the main objectives were achieved, and that the system can generate both STL and G-code reliably in the supported scenarios. At the same time, it also showed that usability and operational robustness still need refinement, especially around onboarding, validation feedback, and runtime configuration. This is not a failure of the approach; rather, it is a natural outcome of the first implementation of a new workflow tool. The foundation is solid; the refinement work is well-scoped.

6.2 Key Contributions

Main contributions of the project are:

- An integrated browser-based pipeline from parametric input to G-code output.
- A modular backend architecture separating routes, generation logic, and slicer services.
- Support for extensibility through user template upload and metadata management.
- Practical multi-part handling and dual preview behavior for complex template cases.

These contributions matter in an educational software engineering context because they combine implementation depth with practical usability outcomes. The project shows that a constrained but purposeful feature set can deliver a lot of value when the architecture is clear and the workflow is carefully designed.

6.3 Answer to Research Questions

The project outcomes support the idea that template-first generation can reduce user friction for common FDM tasks. Bringing model generation and slicing into one platform improved workflow continuity and reduced manual handoff steps. The main trade-off was between keeping the interface simple and offering enough configurability, especially in settings validation and profile management.

The findings support all three research questions, although not with equal strength. The clearest evidence relates to workflow simplification and integrated execution. The area that still needs the most work is the balance between configurability and ease of use, which will require deeper user testing and stronger validation in future versions.

The evaluation results reinforce this conclusion. The platform showed that a template-first workflow can reduce repeated setup effort and support faster iteration, but the benefit depends on clear defaults, stable deployment, and better guidance for less experienced users. In short, the project answers the research questions positively, but within a defined scope rather than as a replacement for general-purpose CAD and slicing workflows.

6.4 Lessons Learned

The project reinforced several engineering lessons:

- Reproducible infrastructure is essential, but runtime path correctness must be verified continuously. Early in development, template paths were hard-coded to local paths; when the system was containerized, paths broke until environment setup was carefully specified. This taught the lesson that infrastructure assumptions are implementation details that must be tested, not just documented.

- API contract clarity between frontend and backend reduces integration defects. By defining request and response models explicitly in FastAPI, many potential bugs were caught at type-checking time rather than at runtime. TypeScript on the frontend enforced similar contracts, making mismatches between frontend and backend obvious immediately rather than manifesting as silent failures where data was sent but not processed.

- Template conventions should be explicit and consistent to support robust parsing and user onboarding. Early template design was relatively free-form; as more templates were added, inconsistency became a maintenance burden. Establishing a formal template metadata schema and Jinja placeholder pattern early would have reduced refactoring work later.

An additional lesson was the value of incremental end-to-end delivery. Building complete vertical slices exposed integration assumptions earlier than isolated component development would have done, which reduced late-stage surprises and improved confidence in system behaviour. For example, by attempting a full workflow in Phase 2 (generating STL), it became apparent immediately that template rendering had assumptions about Jinja whitespace behavior that affected the generated SCAD syntax. Discovering this during isolated SCAD rendering testing would have been possible, but the full end-to-end test made the problem obvious and unmissable.

A practical lesson was also that deployment is not infrastructure; it is part of the application architecture. The clean separation between frontend, backend,

and external tools is conceptually elegant, but that elegance only materializes if the deployment layer correctly wires everything together. Time spent on Docker configuration and environment variable management was necessary engineering work, not optional polish.

6.5 Limitations

The evaluation is bounded by controlled-environment testing, limited user-study breadth, and hardware-dependent performance evidence. These constraints should be considered when generalizing results to broader production contexts.

The limitations do not invalidate the contribution, but they do shape how the results should be interpreted: as evidence of feasibility and practical value within a defined scope rather than as a final industrial benchmark.

The evaluation chapter makes this boundary clear. The platform is functionally successful and architecturally coherent, but it still depends on correct environment setup, sensible slicing settings, and a relatively small set of representative scenarios. This means the project demonstrates a credible proof of concept rather than a fully hardened production system.

6.6 Future Work

Future improvements include:

- richer template validation and parameter type inference,
- broader user-study evaluation with structured usability metrics,
- advanced profile editing and preview-time manufacturability checks,
- persistent job history and export management.

Future development should prioritise both depth and breadth: depth through stronger validation, observability, and benchmarking; breadth through broader template coverage, larger user testing cohorts, and cross-environment deployment trials.

Overall, the project shows that a web-native, template-first strategy is a viable and valuable way to approach practical 3D model generation and print preparation.

6.7 Final Remarks

By integrating established open-source geometry and slicing toolchains behind a focused web interface, the project offers a practical software engineering approach to reducing complexity in day-to-day FDM preparation workflows.

In summary, the project shows that targeted integration can improve usability and reliability in additive manufacturing software workflows when the problem is scoped carefully and evaluated against clear objectives. The platform provides a strong foundation for further refinement, but the evaluation also shows that additional validation, stronger usability support, and better operational resilience are needed before it can be considered a mature production tool.

Bibliography

[1] Vercel. Next.js documentation. https://nextjs.org/docs, 2026. Accessed: 2026-04-15.

[2] FastAPI Contributors. Fastapi documentation. https://fastapi.tiangolo.com/, 2026. Accessed: 2026-04-15.

[3] OpenSCAD Community. Openscad user manual. https://openscad.org/documentation.html, 2026. Accessed: 2026-04-15.

[4] Ultimaker. Curaengine repository. https://github.com/Ultimaker/CuraEngine, 2026. Accessed: 2026-04-15.

[5] Docker Inc. Docker documentation. https://docs.docker.com/, 2026. Accessed: 2026-04-15.

[6] Beck, Kent and others. Manifesto for agile software development. https://agilemanifesto.org/, 2001. Accessed: 2026-04-15.

[7] Bambu Lab. Makerworld. https://makerworld.com/, 2026. Accessed: 2026-04-23.

[8] Physna Inc. Thangs. https://thangs.com/, 2026. Accessed: 2026-04-23.

[9] Prusa Research. Printables. https://www.printables.com/, 2026. Accessed: 2026-04-23.

Appendices

Appendix A

Github Repository

The source code for this project is available on GitHub: `https://github.com/MichalZdziubany/STLGeneration`

Appendix B

Screencast

The link to the screencast is available here: `https://youtu.be/nd61RHoxfcw`